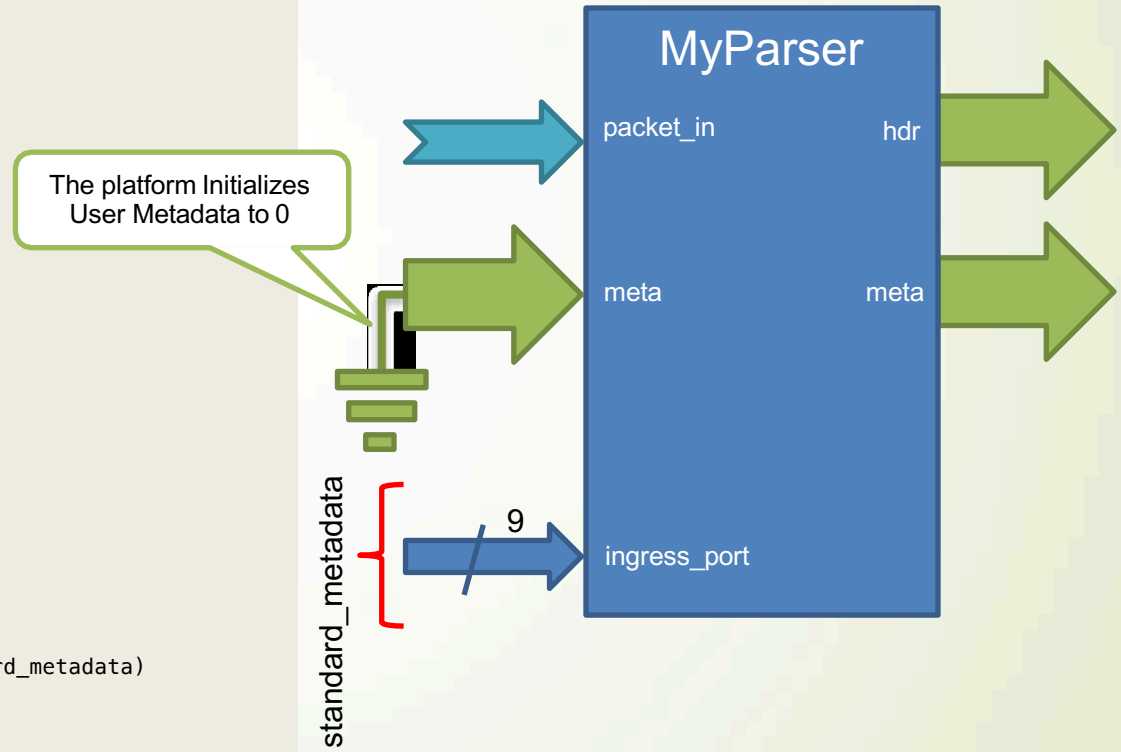
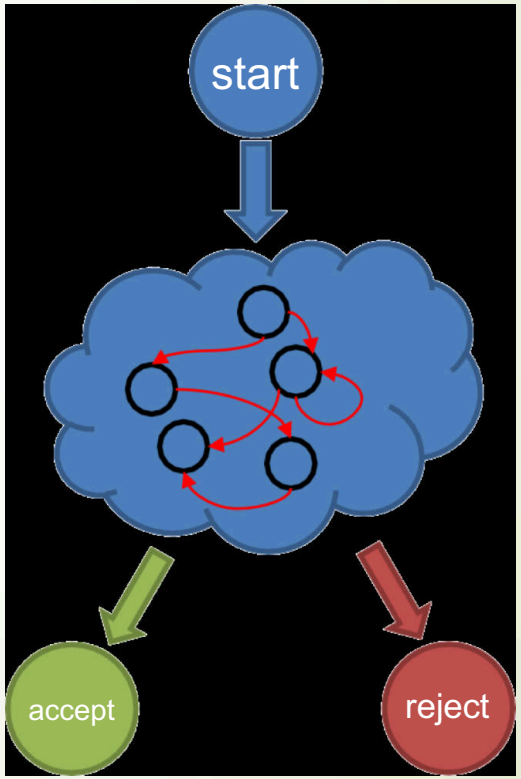
Parser Model (V1 Architecture)

```
#include <core.p4>
#include <v1model.p4>

/* User-defined inputs and outputs */
struct my_headers_t {
    ethernet_t    ethernet;
    vlan_tag_t[2] vlan_tag;
    ipv4_t        ipv4;
    ipv6_t        ipv6;
}

struct my_metadata_t {
    /* Nothing yet */
}

/* System-provided inputs. Others to follow */
struct standard_metadata_t {
    bit<9>   ingress_port;
    . . .
}

/* Parser Declaration */
parser MyParser(packet_in                packet,
                out   my_headers_t       hdr,
                inout my_metadata_t      meta,
                inout standard_metada_t  standard_metadata)
{
   . . .
```



Parsers in P4$_{16}$

- **Parsers are special functions written in a state machine style**
- **Parsers have three predefined states**
  - start
  - accept
  - reject
    - Can be reached explicitly or implicitly
    - What happens in reject state is defined by an architecture
- **Other states are user-defined**



Sample Parser State Machine

```
parser MyParser(packet_in                  packet,
                out   my_headers_t         hdr,
                inout my_metadata_t        meta,
                in    standard_metadata_t  standard_metadata)
{
    state start {
        packet.extract(hdr.ethernet);
        transition select(hdr.ethernet.etherType) {
            0x8100 &&& 0xEFFF : parse_vlan_tag;
            0x0800  : parse_ipv4;
            0x86DD  : parse_ipv6;
            0x0806  : parse_arp;
            default : accept;
        }
    }

    state parse_vlan_tag {
        packet.extract(hdr.vlan_tag.next);
        transition select(hdr.vlan_tag.last.etherType) {
             0x8100 : parse_vlan_tag;
             0x0800 : parse_ipv4;
             0x86DD : parse_ipv6;
             0x0806 : parse_arp;
            default : accept;
        }
    }
```

```
    state parse_ipv4 {
        packet.extract(hdr.ipv4);
        transition select(hdr.ipv4.ihl) {
            0 .. 4: reject;
                 5: accept;
           default: parse_ipv4_options;
    }

    state parse_ipv4_options {
        packet.extract(hdr.ipv4.options,
                       (hdr.ipv4.ihl - 5) << 2);
        transition accept;
    }

    state parse_ipv6 {
        packet.extract(hdr.ipv6);
        transition accept;
    }
```

Lookahead

Example: Typical MPLS Heuristic

```
header ip46_t {    /* Common for both IPv4 and IPv6 */
    bit<4> version;
    bit<4> reserved;
}

state parse_mpls {
    packet.extract(hdr.mpls.next);
    transition select(hdr.mpls.last.bos) {
        0: parse_mpls;
        1: guess_mpls_payload;
    }
}

state guess_mpls_payload {
    transition select(packet.lookahead<ip46_t>().version) {
              4 : parse_inner_ipv4;
              6 : parse_inner_ipv6;
        default : parse_inner_ethernet;
    }
}
```

- **lookahead is a generic method**
  - Compiler generates the method with the proper return type (ipv4_t) at the compile time
- **Returns the packet data without advancing the cursor**
  - The packet length is still checked
- **Much safer and easier to use than bit offsets**

Agenda

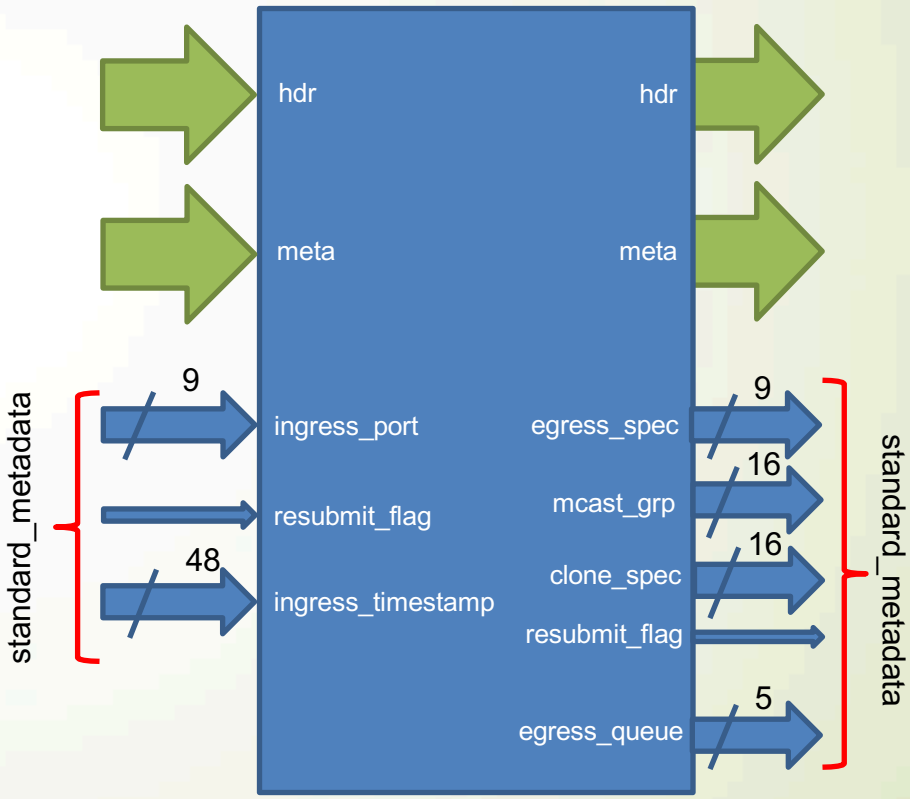
- Why Dataplan Programming
- P4 Overview
- Main Data Types
- Packet Parsing
- Packet Processing
- Packet Deparsing
- Program Structure
- P4 Runtime
- Summary

Controls in P4

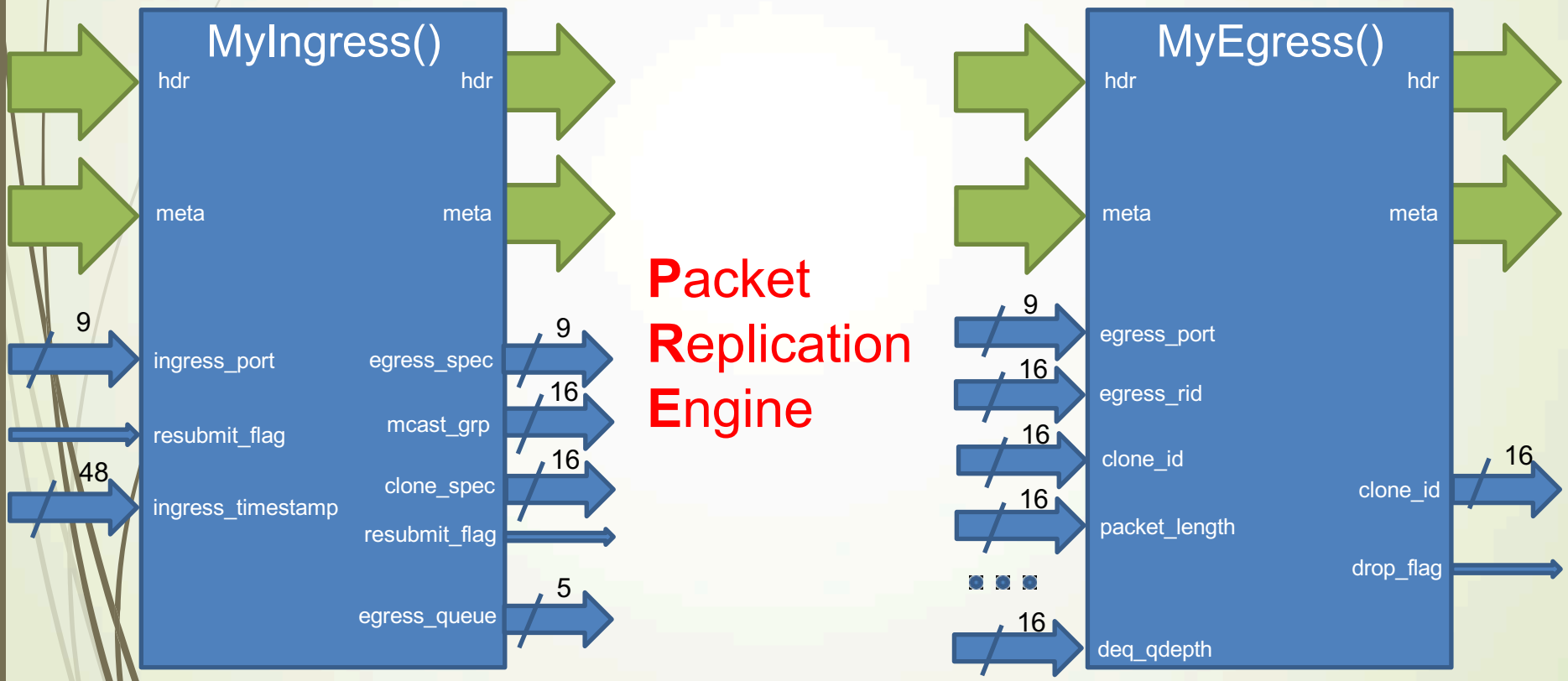
- **Very similar to C functions without loops**
  - Algorithms should be representable as Direct Acyclic Graphs (DAG)
- **Represent all kinds of processing that are expressible as DAG:**
  - Match-Action Pipelines
  - Deparsers
  - Additional processing (checksum updates)
- **Interface with other blocks via user- and architecture-defined data**

Simple Reflector (V1 Architecture)

```
control MyIngress(
    inout my_headers_t          hdr,
    inout my_metadata_t         meta,
    inout standard_metadata_t   standard_metadata)
{
    bit<48> tmp;

    apply {
        tmp = hdr.ethernet.dstAddr;
        hdr.ethernet.dstAddr = hdr.ethernet.srcAddr;
        hdr.ethernet.srcAddr = tmp;

        standard_metadata.egress_spec =
                            standard_metadata.ingress_port;
    }
}
```



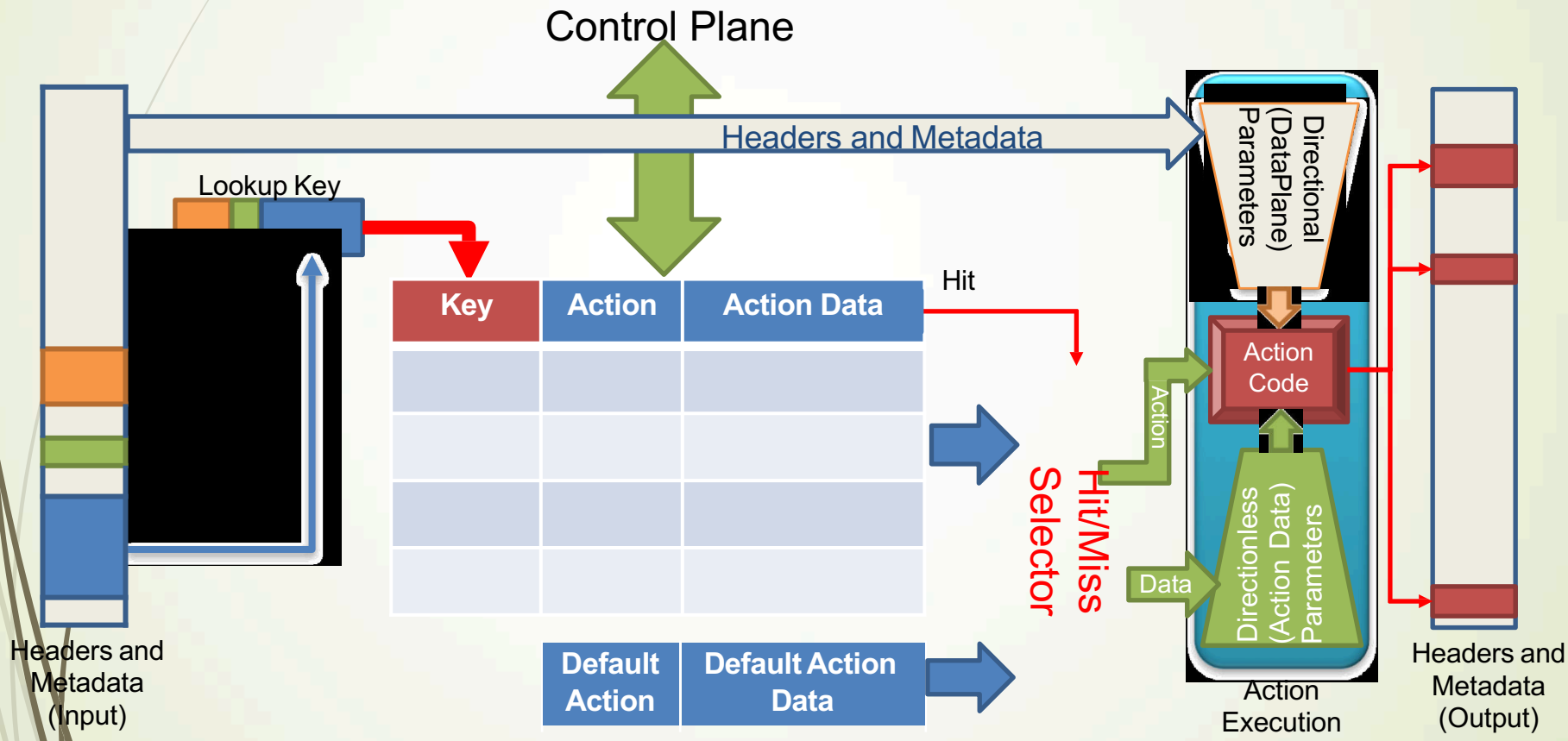
How Does V1 Architecture Work?



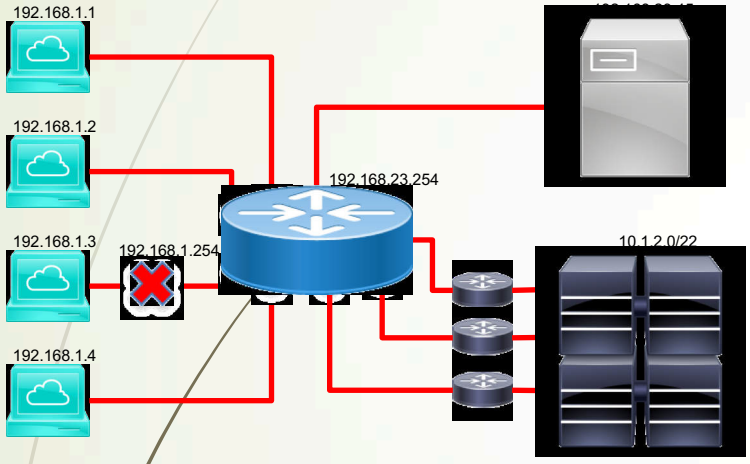
Match-Action Tables

- **The fundamental units of the Match-Action Pipeline**
  - What to match on and match type
  - A list of *possible* actions
  - Additional **properties**
    - Size
    - Default Action
    - Entries
    - etc.
- **Each table contains one or more entries (rows)**
- **An entry contains:**
  - A specific key to match on
  - A **single** action
    - to be executed when a packet matches the entry
  - (Optional) action data

Tables: Match-Action Processing



Example: Basic IPv4 Forwarding



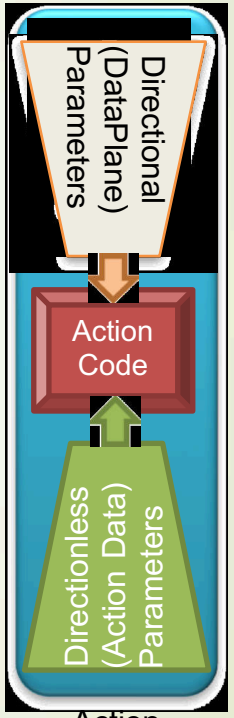
Key	Action	Action Data
192.168.1.1	l3_switch	port= mac_da= mac_sa= vlan=
192.168.1.2	l3_switch	port= mac_da= mac_sa= vlan=…
192.168.1.3	l3_drop	
192.168.1.254	l3_l2_switch	port=
192.168.1.0/24	l3_l2_switch	port=
10.1.2.0/22	l3_switch_ecmp	ecmp_group=

- **Data Plane (P4) Program**
  - Defines the format of the table
    - Key Fields
    - Actions
    - Action Data
  - Performs the lookup
  - Executes the chosen action
- **Control Plane (IP stack, Routing protocols)**
  - Populates table entries with specific information
    - Based on the configuration
    - Based on automatic discovery
    - Based on protocol calculations

Defining Actions for L3 forwarding

```
action l3_switch(bit<9>  port,
                 bit<48> new_mac_da,
                 bit<48> new_mac_sa,
                 bit<12> new_vlan)
{
    /* Forward the packet to the specified port */
    standard_metadata.metadata.egress_spec = port;

    /* L2 Modifications */
    hdr.ethernet.dstAddr   = new_mac_da;
    hdr.ethernet.srcAddr   = mac_sa;
    hdr.vlan_tag[0].vlanid = new_vlan;

    /* IP header modification (TTL decrement) */
    hdr.ipv4.ttl = hdr.ipv4.ttl - 1;
}

action l3_l2_switch(bit<9> port) {
    standard_metadata.metadata.egress_spec = port;
}

action l3_drop() {
    mark_to_drop();
}
```

- Actions can use two types of parameters
  - Directional (from the Data Plane)
  - Directionless (from the Control Plane)
- Actions that are called directly:
  - Only use directional parameters
- Actions used in tables:
  - Typically use direction-less parameters
  - May sometimes use directional parameters too



Action Execution

Match-Action Table

Example: A typical L3 (IPv4) Routing table

```
table ipv4_lpm {
    key = {
        meta.ingress_metadata.vrf    : exac
        hdr.ipv4.dstAddr             : lpm;
    }
    actions = {
        l3_switch;
        l3_l2_switch;
        l3_switch_nexthop(meta.l3.nexthop_info);
        l3_switch_ecmp(meta.l3.nexthop_info);
        l3_drop;  noAction;
    }
    const default_action = l3_l2_switch(CPU_P
    size = 16384;
}
```

Different fields can use different match types

Different fields can use different match types

Prefix length also serves as a priority indicator

vrf	ipv4.dstAddr / prefix		data
1	192.168.1.0 / 24	l3_l2_switch	port_id=3
10	10.0.16.0 / 22	l3_ecmp	ecmp_index=12
1	192.168.0.0 / 16	l3_switch_nexthop	nexthop_index=451
1	0.0.0.0 / 0	l3_switch_nexthop	nexthop_index=1
	DEFAULT	l3_l2_switch	port_id=CPU_PORT

Agenda

- Why Dataplan Programming
- P4 Overview
- Main Data Types
- Packet Parsing
- Packet Processing
- Packet Deparsing
- Program Structure
- P4 Runtime
- Summary